

LAB ASSIGNMENT-15.1

Name: V. Alok

Roll.No: 2303A53003

Batch-46

Task 1: Student Records API

Prompt

Generate a Flask REST API for managing student records with CRUD operations using an in-memory dictionary. Include endpoints for GET, POST, PUT, DELETE and return JSON responses. Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

students = {}

# GET all students
@app.route('/students', methods=['GET']) def
get_students():
    return jsonify(students)

# POST new student
@app.route('/students', methods=['POST']) def
add_student():    data = request.json
    student_id = str(len(students) + 1)
    students[student_id] = data
    return jsonify({"message": "Student added", "id": student_id})

# PUT update student
@app.route('/students/<id>', methods=['PUT'])
def update_student(id):    if id in students:
    students[id] = request.json    return
    jsonify({"message": "Updated"})
    return jsonify({"error": "Student not found"}), 404

# DELETE student
@app.route('/students/<id>', methods=['DELETE'])
```

```

def delete_student(id):
    if id in students:
        del students[id]
        return jsonify({"message": "Deleted"})
    return jsonify({"error": "Student not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

# Test Cases
assert isinstance(students, dict)
students["1"] = {"name": "Akhil"}
assert students["1"]["name"] == "Akhil"
del students["1"]
assert "1" not in students

```

Task 2: Library Book API

Prompt

Create a Flask API for managing books with CRUD operations and PATCH support.

Include error handling for invalid IDs. Code

```
books = {}
```

```

@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

```

```

@app.route('/books', methods=['POST'])
def add_book():
    data = request.json
    book_id = str(len(books) + 1)
    books[book_id] = data
    return jsonify({"id": book_id})

```

```

@app.route('/books/<id>', methods=['GET'])
def get_book(id):
    if id in books:
        return jsonify(books[id])
    return jsonify({"error": "Not found"}), 404

```

```

methods=['PATCH']) def update_book(id):
if id in books:
    books[id].update(request.json)
return jsonify({"message": "Updated"})
    return jsonify({"error": "Not found"}), 404

@app.route('/books/<id>',
methods=['DELETE']) def delete_book(id): if
id in books: del books[id]
    return jsonify({"message": "Deleted"})
return jsonify({"error": "Not found"}), 404 Test

```

Cases

```

books["1"] = {"title": "Python"} assert
books["1"]["title"] == "Python"
books["1"].update({"available":
True}) assert books["1"]["available"]
== True del books["1"] assert "1" not
in books

```

Task 3: Employee Payroll API

Prompt

Build a Flask API to manage employees and salaries with endpoints for listing employees, adding employees, updating salary, and deleting employees. Add docstrings.

Code

```

employees = {}

@app.route('/employees', methods=['GET'])
def get_employees(): """Return all
employees"""
    return jsonify(employees)

@app.route('/employees',
methods=['POST']) def add_employee():
"""Add new employee""" data =
request.json emp_id = str(len(employees)
+ 1) employees[emp_id] = data return
jsonify({"id": emp_id})

```

```
@app.route('/employees/<id>/salary',
methods=['PUT']) def update_salary(id):
    """Update employee salary"""    if id in
employees:
    employees[id]['salary']    = request.json.get('salary')
                                return
 jsonify({"message": "Salary updated"})    return jsonify({"error":
"Not found"}), 404
```

```
@app.route('/employees/<id>',
methods=['DELETE']) def delete_employee(id):
    """Delete employee"""    if id in employees:
del employees[id]
    return jsonify({"message": "Deleted"})
return jsonify({"error": "Not found"}), 404
```

Cases

```
employees["1"] = {"name": "John", "salary":
5000} assert employees["1"]["salary"] == 5000
employees["1"]["salary"] = 6000 assert
employees["1"]["salary"] == 6000 del
employees["1"] assert "1" not in employees
```

Task 4: Online Food Ordering API

Prompt

Generate a Flask REST API for a food ordering system with endpoints for menu listing, placing orders, tracking, updating, and deleting orders. Suggest improvements.

Code

```
menu = {"1": "Pizza", "2": "Burger"} orders
= {}
```

```
@app.route('/menu', methods=['GET']) def
get_menu():
    return jsonify(menu)
```

```
@app.route('/order', methods=['POST'])
def place_order():    data = request.json
order_id = str(len(orders) + 1)
```

```

orders[order_id] = data    return
 jsonify({"order_id": order_id})
 @app.route('/order/<id>',
 methods=['GET']) def get_order(id):
 if id in orders:    return
 jsonify(orders[id])    return
 jsonify({"error": "Not found"}), 404

```

```

 @app.route('/order/<id>', methods=['PUT'])
 def update_order(id):    if id in orders:
 orders[id] = request.json    return
 jsonify({"message": "Updated"})    return
 jsonify({"error": "Not found"}), 404

```

```

 @app.route('/order/<id>',
 methods=['DELETE']) def delete_order(id):
 if id in orders:    del orders[id]    return
 jsonify({"message": "Cancelled"})    return
 jsonify({"error": "Not found"}), 404 Test
 Cases

```

```

orders["1"] = {"item": "Pizza"} assert
orders["1"]["item"] == "Pizza"
orders["1"] = {"item": "Burger"}
assert orders["1"]["item"] ==
"Burger" del orders["1"] assert "1" not
in orders

```